The Oracle execution plan shows an INDEX UNIQUE SCAN—the operation that only traverses the index tree. It fully utilizes the logarithmic scalability of the index to find the entry very quickly—almost independent of the table size.



Tip

The *execution plan* (sometimes *explain plan* or *query plan*) shows the steps the database takes to execute an SQL statement. Appendix A on page 165 explains how to retrieve and read execution plans with other databases.

After accessing the index, the database must do one more step to fetch the queried data (FIRST_NAME, LAST_NAME) from the table storage: the TABLE ACCESS BY INDEX ROWID operation. This operation can become a performance bottleneck—as explained in "Slow Indexes, Part I"—but there is no such risk in connection with an INDEX UNIQUE SCAN. This operation cannot deliver more than one entry so it cannot trigger more than one table access. That means that the ingredients of a slow query are not present with an INDEX UNIQUE SCAN.

Primary Keys without Unique Index

A primary key does not necessarily need a unique index—you can use a non-unique index as well. In that case the Oracle database does not use an INDEX UNIQUE SCAN but instead the INDEX RANGE SCAN operation. Nonetheless, the constraint still maintains the uniqueness of keys so that the index lookup delivers at most one entry.

One of the reasons for using non-unique indexes for a primary keys are *deferrable constraints*. As opposed to regular constraints, which are validated during statement execution, the database postpones the validation of deferrable constraints until the transaction is committed. Deferred constraints are required for inserting data into tables with circular dependencies.

Concatenated Indexes

Even though the database creates the index for the primary key automatically, there is still room for manual refinements if the key consists of multiple columns. In that case the database creates an index on all primary key columns—a so-called *concatenated* index (also known as *multi-column*, *composite* or *combined* index). Note that the column order of a concatenated index has great impact on its usability so it must be chosen carefully.

For the sake of demonstration, let's assume there is a company merger. The employees of the other company are added to our `EMPLOYEES` table so it becomes ten times as large. There is only one problem: the `EMPLOYEE_ID` is not unique across both companies. We need to extend the primary key by an extra identifier—e.g., a subsidiary ID. Thus the new primary key has two columns: the `EMPLOYEE_ID` as before and the `SUBSIDIARY_ID` to reestablish uniqueness.

The index for the new primary key is therefore defined in the following way:

```
CREATE UNIQUE INDEX employee_pk
    ON employees (employee_id, subsidiary_id);
```

A query for a particular employee has to take the full primary key into account—that is, the `SUBSIDIARY_ID` column also has to be used:

```
SELECT first_name, last_name
  FROM employees
 WHERE employee_id   = 123
   AND subsidiary_id = 30;

-------------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |    1 |    2 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |    2 |
|*2 |  INDEX UNIQUE SCAN          | EMPLOYEES_PK |    1 |    1 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("EMPLOYEE_ID"=123 AND "SUBSIDIARY_ID"=30)
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Whenever a query uses the complete primary key, the database can use an INDEX UNIQUE SCAN—no matter how many columns the index has. But what happens when using only one of the key columns, for example, when searching all employees of a subsidiary?

```
SELECT first_name, last_name
  FROM employees
 WHERE subsidiary_id = 20;

-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |  106 |  478 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |  106 |  478 |
-----------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter("SUBSIDIARY_ID"=20)
```

The execution plan reveals that the database does not use the index. Instead it performs a FULL TABLE SCAN. As a result the database reads the entire table and evaluates every row against the **where** clause. The execution time grows with the table size: if the table grows tenfold, the FULL TABLE SCAN takes ten times as long. The danger of this operation is that it is often fast enough in a small development environment, but it causes serious performance problems in production.

Full Table Scan

The operation TABLE ACCESS FULL, also known as *full table scan*, can be the most efficient operation in some cases anyway, in particular when retrieving a large part of the table.

This is partly due to the overhead for the index lookup itself, which does not happen for a TABLE ACCESS FULL operation. This is mostly because an index lookup reads one block after the other as the database does not know which block to read next until the current block has been processed. A FULL TABLE SCAN must get the entire table anyway so that the database can read larger chunks at a time (*multi block read*). Although the database reads more data, it might need to execute fewer read operations.

The database does not use the index because it cannot use single columns from a concatenated index arbitrarily. A closer look at the index structure makes this clear.

A concatenated index is just a B-tree index like any other that keeps the indexed data in a sorted list. The database considers each column according to its position in the index definition to sort the index entries. The first column is the primary sort criterion and the second column determines the order only if two entries have the same value in the first column and so on.

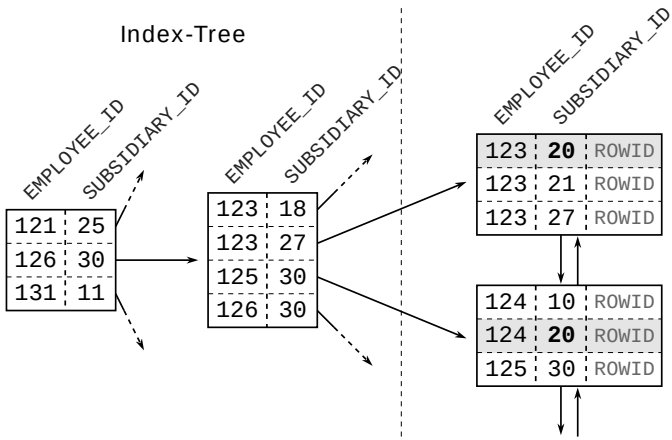


Important

A concatenated index is *one index across multiple columns*.

The ordering of a two-column index is therefore like the ordering of a telephone directory: it is first sorted by surname, then by first name. That means that a two-column index does not support searching on the second column alone; that would be like searching a telephone directory by first name.

Figure 2.1. Concatenated Index



The index excerpt in Figure 2.1 shows that the entries for subsidiary 20 are not stored next to each other. It is also apparent that there are no entries with `SUBSIDIARY_ID = 20` in the tree, although they exist in the leaf nodes. The tree is therefore useless for this query.



Tip

Visualizing an index helps in understanding what queries the index supports. You can query the database to retrieve the entries in index order (SQL:2008 syntax, see page 144 for proprietary solutions using LIMIT, TOP or ROWNUM):

```
SELECT <INDEX COLUMN LIST>
  FROM <TABLE>
 ORDER BY <INDEX COLUMN LIST>
 FETCH FIRST 100 ROWS ONLY;
```

If you put the index definition and table name into the query, you will get a sample from the index. Ask yourself if the requested rows are clustered in a central place. If not, the index tree cannot help find that place.

We could, of course, add another index on SUBSIDIARY_ID to improve query speed. There is however a better solution—at least if we assume that searching on EMPLOYEE_ID alone does not make sense.

We can take advantage of the fact that the first index column is always usable for searching. Again, it is like a telephone directory: you don't need to know the first name to search by last name. The trick is to reverse the index column order so that the SUBSIDIARY_ID is in the first position:

```
CREATE UNIQUE INDEX EMPLOYEES_PK
    ON EMPLOYEES (SUBSIDIARY_ID, EMPLOYEE_ID);
```

Both columns together are still unique so queries with the full primary key can still use an INDEX UNIQUE SCAN but the sequence of index entries is entirely different. The SUBSIDIARY_ID has become the primary sort criterion. That means that all entries for a subsidiary are in the index consecutively so the database can use the B-tree to find their location.



Important

The most important consideration when defining a concatenated index is how to choose the column order so it can be used as often as possible.

The execution plan confirms that the database uses the "reversed" index. The `SUBSIDIARY_ID` alone is not unique anymore so the database must follow the leaf nodes in order to find all matching entries: it is therefore using the `INDEX RANGE SCAN` operation.

```
-------------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |  106 |   75 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |  106 |   75 |
|*2 |  INDEX RANGE SCAN           | EMPLOYEE_PK  |  106 |    2 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("SUBSIDIARY_ID"=20)
```

In general, a database can use a concatenated index when searching with the leading (leftmost) columns. An index with three columns can be used when searching for the first column, when searching with the first two columns together, and when searching using all columns.

Even though the two-index solution delivers very good **`select`** performance as well, the single-index solution is preferable. It not only saves storage space, but also the maintenance overhead for the second index. The fewer indexes a table has, the better the **`insert`**, **`delete`** and **`update`** performance.

To define an optimal index you must understand more than just how indexes work—you must also know how the application queries the data. This means you have to know the column combinations that appear in the **`where`** clause.

Defining an optimal index is therefore very difficult for external consultants because they don't have an overview of the application's access paths. Consultants can usually consider one query only. They do not exploit the extra benefit the index could bring for other queries. Database administrators are in a similar position as they might know the database schema but do not have deep insight into the access paths.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

The only place where the technical database knowledge meets the functional knowledge of the business domain is the development department. Developers have a feeling for the data and know the access path. They can properly index to get the best benefit for the overall application without much effort.

Slow Indexes, Part II

The previous section explained how to gain additional benefits from an existing index by changing its column order, but the example considered only two SQL statements. Changing an index, however, may affect all queries on the indexed table. This section explains the way databases pick an index and demonstrates the possible side effects when changing existing indexes.

The adopted `EMPLOYEE_PK` index improves the performance of all queries that search by subsidiary only. It is however usable for all queries that search by `SUBSIDIARY_ID`—regardless of whether there are any additional search criteria. That means the index becomes usable for queries that used to use another index with another part of the **`where`** clause. In that case, if there are multiple access paths available it is the optimizer's job to choose the best one.

The Query Optimizer

The query optimizer, or query planner, is the database component that transforms an SQL statement into an execution plan. This process is also called *compiling* or *parsing*. There are two distinct optimizer types.

Cost-based optimizers (CBO) generate many execution plan variations and calculate a *cost* value for each plan. The cost calculation is based on the operations in use and the estimated row numbers. In the end the cost value serves as the benchmark for picking the "best" execution plan.

Rule-based optimizers (RBO) generate the execution plan using a hard-coded rule set. Rule based optimizers are less flexible and are seldom used today.

Changing an index might have unpleasant side effects as well. In our example, it is the internal telephone directory application that has become very slow since the merger. The first analysis identified the following query as the cause for the slowdown:

```
SELECT first_name, last_name, subsidiary_id, phone_number
  FROM employees
 WHERE last_name  = 'WINAND'
   AND subsidiary_id = 30;
```

The execution plan is:

Example 2.1. Execution Plan with Revised Primary Key Index

```
-------------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |    1 |   30 |
|*1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |   30 |
|*2 |  INDEX RANGE SCAN           | EMPLOYEES_PK |   40 |    2 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
  1 - filter("LAST_NAME"='WINAND')
  2 - access("SUBSIDIARY_ID"=30)
```

The execution plan uses an index and has an overall cost value of 30. So far, so good. It is however suspicious that it uses the index we just changed—that is enough reason to suspect that our index change caused the performance problem, especially when bearing the old index definition in mind—it started with the `EMPLOYEE_ID` column which is not part of the **`where`** clause at all. The query could not use that index before.

For further analysis, it would be nice to compare the execution plan before and after the change. To get the original execution plan, we could just deploy the old index definition again, however most databases offer a simpler method to prevent using an index for a specific query. The following example uses an Oracle *optimizer hint* for that purpose.

```
SELECT /*+ NO_INDEX(EMPLOYEES EMPLOYEE_PK) */
       first_name, last_name, subsidiary_id, phone_number
  FROM employees
 WHERE last_name  = 'WINAND'
   AND subsidiary_id = 30;
```

The execution plan that was presumably used before the index change did not use an index at all:

```
-------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-------------------------------------------------
|  0 | SELECT STATEMENT  |           |    1 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    1 |  477 |
-------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter("LAST_NAME"='WINAND' AND "SUBSIDIARY_ID"=30)
```

Even though the TABLE ACCESS FULL must read and process the entire table, it seems to be faster than using the index in this case. That is particularly unusual because the query matches one row only. Using an index to find a single row should be much faster than a full table scan, but in this case it is not. The index seems to be slow.

In such cases it is best to go through each step of the troublesome execution plan. The first step is the INDEX RANGE SCAN on the EMPLOYEES_PK index. That index does not cover the LAST_NAME column—the INDEX RANGE SCAN can consider the SUBSIDIARY_ID filter only; the Oracle database shows this in the "Predicate Information" area—entry "2" of the execution plan. There you can see the conditions that are applied for each operation.



Tip

Appendix A, "*Execution Plans*", explains how to find the "Predicate Information" for other databases.

The INDEX RANGE SCAN with operation ID 2 (Example 2.1 on page 19) applies only the SUBSIDIARY_ID=30 filter. That means that it traverses the index tree to find the first entry for SUBSIDIARY_ID 30. Next it follows the leaf node chain to find all other entries for that subsidiary. The result of the INDEX RANGE SCAN is a list of ROWIDs that fulfill the SUBSIDIARY_ID condition: depending on the subsidiary size, there might be just a few ones or there could be many hundreds.

The next step is the TABLE ACCESS BY INDEX ROWID operation. It uses the ROWIDs from the previous step to fetch the rows—all columns—from the table. Once the LAST_NAME column is available, the database can evaluate the remaining part of the **where** clause. That means the database has to fetch all rows for SUBSIDIARY_ID=30 before it can apply the LAST_NAME filter.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

The statement's response time does not depend on the result set size but on the number of employees in the particular subsidiary. If the subsidiary has just a few members, the INDEX RANGE SCAN provides better performance. Nonetheless a TABLE ACCESS FULL can be faster for a huge subsidiary because it can read large parts from the table in one shot (see "Full Table Scan" on page 13).

The query is slow because the index lookup returns many ROWIDs—one for each employee of the original company—and the database must fetch them individually. It is the perfect combination of the two ingredients that make an index slow: the database reads a wide index range and has to fetch many rows individually.

Choosing the best execution plan depends on the table's data distribution as well so the optimizer uses statistics about the contents of the database. In our example, a histogram containing the distribution of employees over subsidiaries is used. This allows the optimizer to estimate the number of rows returned from the index lookup—the result is used for the cost calculation.

Statistics

A cost-based optimizer uses statistics about tables, columns, and indexes. Most statistics are collected on the column level: the number of distinct values, the smallest and largest values (data range), the number of NULL occurrences and the column histogram (data distribution). The most important statistical value for a table is its size (in rows and blocks).

The most important index statistics are the tree depth, the number of leaf nodes, the number of distinct keys and the clustering factor (see Chapter 5, "*Clustering Data*").

The optimizer uses these values to estimate the selectivity of the **where** clause predicates.

If there are no statistics available—for example because they were deleted—the optimizer uses default values. The default statistics of the Oracle database suggest a small index with medium selectivity. They lead to the estimate that the INDEX RANGE SCAN will return 40 rows. The execution plan shows this estimation in the Rows column (again, see Example 2.1 on page 19). Obviously this is a gross underestimate, as there are 1000 employees working for this subsidiary.

If we provide correct statistics, the optimizer does a better job. The following execution plan shows the new estimation: 1000 rows for the INDEX RANGE SCAN. Consequently it calculated a higher cost value for the subsequent table access.

```
-------------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |    1 |  680 |
|*1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |  680 |
|*2 |  INDEX RANGE SCAN           | EMPLOYEES_PK | 1000 |    4 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
  1 - filter("LAST_NAME"='WINAND')
  2 - access("SUBSIDIARY_ID"=30)
```

The cost value of 680 is even higher than the cost value for the execution plan using the FULL TABLE SCAN (477, see page 20). The optimizer will therefore automatically prefer the FULL TABLE SCAN.

This example of a slow index should not hide the fact that proper indexing is the best solution. Of course searching on last name is best supported by an index on LAST_NAME:

```
CREATE INDEX emp_name ON employees (last_name);
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Using the new index, the optimizer calculates a cost value of 3:

Example 2.2. Execution Plan with Dedicated Index

```
-----------------------------------------------------------------
| Id | Operation                    | Name      | Rows | Cost |
-----------------------------------------------------------------
|  0 | SELECT STATEMENT             |           |    1 |    3 |
|* 1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES |    1 |    3 |
|* 2 |   INDEX RANGE SCAN           | EMP_NAME  |    1 |    1 |
-----------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter("SUBSIDIARY_ID"=30)
   2 - access("LAST_NAME"='WINAND')
```

The index access delivers—according to the optimizer's estimation—one row only. The database thus has to fetch only that row from the table: this is definitely faster than a `FULL TABLE SCAN`. A properly defined index is still better than the original full table scan.

The two execution plans from Example 2.1 (page 19) and Example 2.2 are almost identical. The database performs the same operations and the optimizer calculated similar cost values, nevertheless the second plan performs much better. The efficiency of an `INDEX RANGE SCAN` may vary over a wide range—especially when followed by a table access. Using an index does not automatically mean a statement is executed in the best way possible.

Functions

The index on LAST_NAME has improved the performance considerably, but it requires you to search using the same case (upper/lower) as is stored in the database. This section explains how to lift this restriction without a decrease in performance.



Note

MySQL 5.6 does not support function-based indexing as described below. As an alternative, virtual columns were planned for MySQL 6.0 but were introduced in MariaDB 5.2 only.

Case-Insensitive Search Using UPPER or LOWER

Ignoring the case in a **where** clause is very simple. You can, for example, convert both sides of the comparison to all caps notation:

```
SELECT first_name, last_name, phone_number
  FROM employees
 WHERE UPPER(last_name) = UPPER('winand');
```

Regardless of the capitalization used for the search term or the LAST_NAME column, the UPPER function makes them match as desired.



Note

Another way for case-insensitive matching is to use a different "collation". The default collations used by SQL Server and MySQL do not distinguish between upper and lower case letters—they are case-insensitive by default.

The logic of this query is perfectly reasonable but the execution plan is not:

```
-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |   10 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |   10 |  477 |
-----------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter(UPPER("LAST_NAME")='WINAND')
```

It is a return of our old friend the full table scan. Although there is an index on LAST_NAME, it is unusable—because the search is *not* on LAST_NAME but on UPPER(LAST_NAME). From the database's perspective, that's something *entirely different*.

This is a trap we all might fall into. We recognize the relation between LAST_NAME and UPPER(LAST_NAME) instantly and expect the database to "see" it as well. In reality the optimizer's view is more like this:

```
SELECT first_name, last_name, phone_number
  FROM employees
 WHERE BLACKBOX(...) = 'WINAND';
```

The UPPER function is just a black box. The parameters to the function are not relevant because there is no general relationship between the function's parameters and the result.



Tip

Replace the function name with BLACKBOX to understand the optimizer's point of view.

Compile Time Evaluation

The optimizer can evaluate the expression on the right-hand side during "compile time" because it has all the input parameters. The Oracle execution plan ("Predicate Information" section) therefore only shows the upper case notation of the search term. This behavior is very similar to a compiler that evaluates constant expressions at compile time.

To support that query, we need an index that covers the actual search term. That means we do not need an index on LAST_NAME but on UPPER(LAST_NAME):

```
CREATE INDEX emp_up_name
    ON employees (UPPER(last_name));
```

An index whose definition contains functions or expressions is a so-called *function-based index (FBI)*. Instead of copying the column data directly into the index, a function-based index applies the function first and puts the result into the index. As a result, the index stores the names in all caps notation.

The database can use a function-based index if the *exact* expression of the index definition appears in an SQL statement—like in the example above. The execution plan confirms this:

```
-------------------------------------------------------------
|Id |Operation                    | Name        | Rows | Cost |
-------------------------------------------------------------
| 0 |SELECT STATEMENT             |             |  100 |   41 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES   |  100 |   41 |
|*2 |  INDEX RANGE SCAN           | EMP_UP_NAME |   40 |    1 |
-------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
  2 - access(UPPER("LAST_NAME")='WINAND')
```

It is a regular INDEX RANGE SCAN as described in Chapter 1. The database traverses the B-tree and follows the leaf node chain. There are no dedicated operations or keywords for function-based indexes.



Warning

Sometimes ORM tools use UPPER and LOWER without the developer's knowledge. Hibernate, for example, injects an implicit LOWER for case-insensitive searches.

The execution plan is not yet the same as it was in the previous section without UPPER; the row count estimate is too high. It is particularly strange that the optimizer expects to fetch more rows from the table than the INDEX RANGE SCAN delivers in the first place. How can it fetch 100 rows from the table if the preceding index scan returned only 40 rows? The answer is that it can not. Contradicting estimates like this often indicate problems with the statistics. In this particular case it is because the Oracle database

does not update the table statistics when creating a new index (see also “Oracle Statistics for Function-Based Indexes” on page 28).

After updating the statistics, the optimizer calculates more accurate estimates:

```
-------------------------------------------------------------
|Id |Operation                    | Name        | Rows | Cost |
-------------------------------------------------------------
| 0 |SELECT STATEMENT             |             |    1 |    3 |
| 1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES   |    1 |    3 |
|*2 |   INDEX RANGE SCAN          | EMP_UP_NAME |    1 |    1 |
-------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access(UPPER("LAST_NAME")='WINAND')
```



NOTE

The so-called “extended statistics” on expressions and column groups were introduced with Oracle release 11*g*.

Although the updated statistics do not improve execution performance in this case—the index was properly used anyway—it is always a good idea to check the optimizer’s estimates. The number of rows processed for each operation (cardinality estimate) is a particularly important figure that is also shown in SQL Server and PostgreSQL execution plans.



TIP

Appendix A, “*Execution Plans*”, describes the row count estimates in SQL Server and PostgreSQL execution plans.

SQL Server does not support function-based indexes as described but it does offer computed columns that can be used instead. To make use of this, you have to first add a computed column to the table that can be indexed afterwards:

```
ALTER TABLE employees ADD last_name_up AS UPPER(last_name);
CREATE INDEX emp_up_name ON employees (last_name_up);
```

SQL Server is able to use this index whenever the indexed expression appears in the statement. You do not need to rewrite your query to use the computed column.

Oracle Statistics for Function-Based Indexes

The Oracle database maintains the information about the number of distinct column values as part of the table statistics. These figures are reused if a column is part of multiple indexes.

Statistics for a function-based index (FBI) are also kept on table level as *virtual columns*. Although the Oracle database collects the *index statistics* for new indexes automatically (since release 10*g*), it does not update the *table statistics*. For this reason, the Oracle documentation recommends updating the table statistics after creating a function-based index:

> After creating a function-based index, collect statistics on both the index and its base table using the `DBMS_STATS` package. Such statistics will enable Oracle Database to correctly decide when to use the index.
>
> —Oracle Database SQL Language Reference

My personal recommendation goes even further: after every index change, update the statistics for the base table and all its indexes. That might, however, also lead to unwanted side effects. Coordinate this activity with the database administrators (DBAs) and make a backup of the original statistics.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

User-Defined Functions

Function-based indexing is a very generic approach. Besides functions like UPPER you can also index expressions like A + B and even use user-defined functions in the index definition.

There is one important exception. It is, for example, not possible to refer to the current time in an index definition, neither directly nor indirectly, as in the following example.

```
CREATE FUNCTION get_age(date_of_birth DATE)
RETURN NUMBER
AS
BEGIN
  RETURN
    TRUNC(MONTHS_BETWEEN(SYSDATE, date_of_birth)/12);
END;
/
```

The function GET_AGE uses the current date (SYSDATE) to calculate the age based on the supplied date of birth. You can use this function in all parts of an SQL query, for example in **select** and the **where** clauses:

```
SELECT first_name, last_name, get_age(date_of_birth)
  FROM employees
 WHERE get_age(date_of_birth) = 42;
```

The query lists all 42-year-old employees. Using a function-based index is an obvious idea for optimizing this query, but you cannot use the function GET_AGE in an index definition because it is not *deterministic*. That means the result of the function call is not fully determined by its parameters. Only functions that always return the same result for the same parameters—functions that are deterministic—can be indexed.

The reason behind this limitation is simple. When inserting a new row, the database calls the function and stores the result in the index and there it stays, unchanged. There is no periodic process that updates the index. The database updates the indexed age only when the date of birth is changed by an **update** statement. After the next birthday, the age that is stored in the index will be wrong.

Besides *being* deterministic, PostgreSQL and the Oracle database require functions to be *declared* to be deterministic when used in an index so you have to use the keyword `DETERMINISTIC` (Oracle) or `IMMUTABLE` (PostgreSQL).



Caution

PostgreSQL and the Oracle database trust the `DETERMINISTIC` or `IMMUTABLE` declarations—that means they trust the developer.

You can declare the `GET_AGE` function to be deterministic and use it in an index definition. Regardless of the declaration, it will *not* work as intended because the age stored in the index will not increase as the years pass; the employees will not get older—at least not in the index.

Other examples for functions that cannot be "indexed" are random number generators and functions that depend on environment variables.



Think about it

How can you still use an index to optimize a query for all 42-year-old employees?

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Over-Indexing

If the concept of function-based indexing is new to you, you might be tempted to just index everything, but this is in fact the very last thing you should do. The reason is that every index causes ongoing maintenance. Function-based indexes are particularly troublesome because they make it very easy to create *redundant indexes*.

The case-insensitive search from above could be implemented with the `LOWER` function as well:

```
SELECT first_name, last_name, phone_number
  FROM employees
 WHERE LOWER(last_name) = LOWER('winand');
```

A single index cannot support both methods of ignoring the case. We could, of course, create a second index on `LOWER(last_name)` for this query, but that would mean the database has to maintain two indexes for each **`insert`**, **`update`**, and **`delete`** statement (see also Chapter 8, "*Modifying Data*"). To make one index suffice, you should consistently use the same function throughout your application.



Tip

Unify the access path so that one index can be used by several queries.



Tip

Always aim to index the original data as that is often the most useful information you can put into an index.